

Dot Net Programming Long Answers

1. .NET Framework Architecture

- .NET Framework is a platform for developing and running applications.
- Main idea: source code is compiled into Intermediate Language (IL), then CLR runs it.
- It supports many languages like C#, VB.NET, F#.

Diagram

```
+-----+
|  Applications  |
+-----+
      |
+-----+
|  Class Library  |
+-----+
      |
+-----+
|      CLR      |
| JIT, GC, Security|
+-----+
      |
+-----+
|  Operating Sys  |
+-----+
```

Main parts

- **CLR:** Executes IL code and manages memory.
- **CTS:** Defines common data types for all languages.
- **CLS:** Rules that languages follow for interoperability.
- **JIT:** Converts IL into machine code at runtime.
- **Assemblies:** Deployment units like .exe and .dll.
- **Garbage Collection:** Frees unused memory automatically.

Key points

- Language-independent execution.
- Security, exception handling, threading.
- Automatic memory management.

2. Inheritance in C#

- Inheritance means one class acquires properties and methods of another class.
- It reduces code duplication and supports reusability.
- C# supports single, multilevel, and hierarchical inheritance.
- Multiple inheritance is not allowed with classes, but possible through interfaces.

Multilevel inheritance

```
using System;
class A { public void ShowA() { Console.WriteLine("A"); } }
class B : A { public void ShowB() { Console.WriteLine("B"); } }
class C : B { public void ShowC() { Console.WriteLine("C"); } }
class Program {
    static void Main() {
        C obj = new C();
        obj.ShowA(); obj.ShowB(); obj.ShowC();
    }
}
```

Multiple inheritance using interfaces

```
using System;
interface I1 { void Print1(); }
interface I2 { void Print2(); }
class Test : I1, I2 {
    public void Print1() { Console.WriteLine("Interface 1"); }
    public void Print2() { Console.WriteLine("Interface 2"); }
}
class Program {
    static void Main() {
        Test t = new Test();
        t.Print1();
        t.Print2();
    }
}
```

3. Delegates, Events, Lambda Expressions, Generic Delegates

Delegates

- Delegate is a type-safe function pointer.
- It stores reference to a method.

```
using System;
delegate void MyDel(string msg);
class Program {
    static void Display(string msg) { Console.WriteLine(msg); }
    static void Main() {
```

```

        MyDel d = Display;
        d("Hello Delegate");
    }
}

```

Events

- Events are built on delegates.
- Used for notification.

```

using System;
class Alarm {
    public event Action Ring;
    public void Start() { Ring?.Invoke(); }
}
class Program {
    static void Main() {
        Alarm a = new Alarm();
        a.Ring += () => Console.WriteLine("Alarm rang!");
        a.Start();
    }
}

```

Lambda expressions

- Short anonymous function syntax.

```

Func<int, int> square = x => x * x;
Console.WriteLine(square(5));

```

Generic delegates

- Delegate with type parameter.

```

Func<int, int, int> add = (a, b) => a + b;
Console.WriteLine(add(2, 3));

```

4. Operator Overloading and Indexers

Operator overloading

- Gives custom meaning to operators for user-defined types.

```

using System;
class Complex {
    public int R, I;
    public Complex(int r, int i) { R = r; I = i; }
    public static Complex operator +(Complex a, Complex b) {
        return new Complex(a.R + b.R, a.I + b.I);
    }
}

```

```

    }
    public void Show() { Console.WriteLine($"{R} + {I}i"); }
}
class Program {
    static void Main() {
        Complex c1 = new Complex(2, 3);
        Complex c2 = new Complex(4, 5);
        Complex c3 = c1 + c2;
        c3.Show();
    }
}

```

Indexers

- Allow object to be used like an array.

```

using System;
class StudentMarks {
    int[] marks = new int[5];
    public int this[int i] {
        get { return marks[i]; }
        set { marks[i] = value; }
    }
}
class Program {
    static void Main() {
        StudentMarks s = new StudentMarks();
        s[0] = 90;
        Console.WriteLine(s[0]);
    }
}

```

5. Exception Handling in C#

- Exception handling prevents program crash.
- Main keywords: try, catch, finally, throw.

IndexOutOfRangeException

```

using System;
class Program {
    static void Main() {
        try {
            int[] a = {1, 2};
            Console.WriteLine(a[5]);
        } catch (IndexOutOfRangeException e) {
            Console.WriteLine(e.Message);
        }
    }
}

```

InvalidCastException

```
using System;
class Program {
    static void Main() {
        try {
            object x = "Hello";
            int y = (int)x;
        } catch (InvalidCastException e) {
            Console.WriteLine(e.Message);
        }
    }
}
```

Custom Exception

```
using System;
class MyException : Exception {
    public MyException(string msg) : base(msg) { }
}
class Program {
    static void Main() {
        try {
            throw new MyException("Custom error");
        } catch (MyException e) {
            Console.WriteLine(e.Message);
        }
    }
}
```

6. LINQ in Detail

- LINQ means Language Integrated Query.
- Used to query collections in C#.
- Query syntax is easy to read.

Employee example

```
using System;
using System.Linq;
using System.Collections.Generic;
class Employee {
    public int Id { get; set; }
    public string Name { get; set; }
    public int Salary { get; set; }
}
class Program {
    static void Main() {
        List<Employee> emp = new List<Employee> {
            new Employee{Id=1, Name="Ram", Salary=30000},
        }
    }
}
```

```

        new Employee{Id=2, Name="Sita", Salary=45000},
        new Employee{Id=3, Name="Hari", Salary=25000}
    };

    var q1 = from e in emp select e.Name;
    var q2 = from e in emp where e.Salary > 30000 select e;
    var q3 = from e in emp orderby e.Salary descending select e;
    bool hasSita = emp.Select(e => e.Name).Contains("Sita");

    foreach (var n in q1) Console.WriteLine(n);
    foreach (var e in q2) Console.WriteLine(e.Name);
    foreach (var e in q3) Console.WriteLine(e.Name);
    Console.WriteLine(hasSita);
}
}

```

Operators

- **Select:** projects data.
- **Where:** filters data.
- **OrderBy:** sorts data.
- **Contains:** checks existence.

7. ASP.NET Registration Form and Display on Another Page

- One page collects user input.
- Another page shows the submitted data.
- Data is usually transferred using QueryString, Session, or Server.Transfer.

Simple flow

Registration Page -> Save data -> Display Page

Example idea

- Register.aspx: name, email, password.
- On submit, store values in Session.
- Display.aspx: read session values and show them.

Short explanation

- ASP.NET uses server controls like TextBox, Button, Label.
- PostBack sends data to server.
- Session is safe for page-to-page data sharing.

8. ASP.NET Validation Controls

- Validation controls check user input before form submission.
- They improve correctness and user experience.

Controls

- **RequiredFieldValidator**: checks empty field.
- **RangeValidator**: checks if value is within range.
- **CompareValidator**: compares two inputs or values.
- **RegularExpressionValidator**: checks format like email.
- **CustomValidator**: user-defined validation logic.
- **ValidationSummary**: shows all errors together.

Short example flow

```
User Input -> Validators -> Valid? -> Submit
                    -> Invalid? -> Error Message
```

9. ADO.NET Database Connection, Insert, Select

- ADO.NET is used to connect C# with databases.
- Common objects: Connection, Command, DataReader.

Program

```
using System;
using System.Data.SqlClient;
class Program {
    static void Main() {
        string cs = "your_connection_string";
        using (SqlConnection con = new SqlConnection(cs)) {
            con.Open();

            SqlCommand insert = new SqlCommand("INSERT INTO Student(Id,Name) VALUES(:", con);
            insert.ExecuteNonQuery();

            SqlCommand select = new SqlCommand("SELECT * FROM Student", con);
            SqlDataReader dr = select.ExecuteReader();
            while (dr.Read()) {
                Console.WriteLine(dr[0] + " " + dr[1]);
            }
        }
    }
}
```

Key points

- `Open()` opens connection.
- `ExecuteNonQuery()` for insert/update/delete.
- `ExecuteReader()` for select.

10. Object-Oriented Features in C#

- C# supports core OOP features strongly.
- Important features: constructors, properties, interfaces, virtual methods, `this`, `base`.

Short examples

```
using System;
interface IPrint { void Show(); }
class A {
    public int x;
    public A(int x) { this.x = x; }
    public virtual void Show() { Console.WriteLine("Base"); }
}
class B : A, IPrint {
    public B(int x) : base(x) { }
    public override void Show() { Console.WriteLine("Derived " + x); }
    public void Print() { Console.WriteLine("Interface"); }
}
```

Notes

- **Constructor:** initializes object.
- **Property:** controlled access to field.
- **Interface:** contract.
- **Virtual method:** can be overridden.
- **this:** current object.
- **base:** parent class member.